

Đặc tả kiến trúc 3 mô hình — Tuần 3, Ngày 4-5

(Nội dung này dùng trực tiếp cho mục 4 của BaoCao_Tuan03.docx, viết ở Ngày 6)

1. Bảng đặc tả tổng quan

Tầng	Input	Layers	Loss	Optimizer	Output
1 — Random Forest	vector (16,) chuẩn hóa (Pipeline 1)	RandomForestClassifier(n_estimators=200, max_depth=None, class_weight='balanced')	— (không dùng gradient-based loss)	—	predict_proba nhân (0/1) + xác suất
1 — SVM	vector (16,) chuẩn hóa (Pipeline 1)	SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, class_weight='balanced')	hinge loss (nội tại scikit-learn)	—	nhân (0/1) + xác suất
2 — LSTM	sequence (250,) index (Pipeline 2)	Embedding(96,64) LSTM(64, return_sequences=False) Dropout(0.2) Dense(1, activation='sigmoid')	binary_crossentropy	Adam(learning_rate=0.001)	xác suất (sigmoid)
3 — LSTM+Attention	sequence (250,) index (Pipeline 2)	Embedding(96,64) LSTM(64, return_sequences=True) Attention(additive, tự cài đặt) Dropout(0.2) Dense(1, activation='sigmoid')	binary_crossentropy	Adam(learning_rate=0.001)	xác suất (sigmoid) + attention_weights (250,)

2. Chi tiết Tầng 1

Random Forest:

- `n_estimators=200` — chọn giữa khoảng gợi ý 100-300 trong kế hoạch, cân bằng giữa độ ổn định và thời gian huấn luyện với quy mô ~24.724 dòng train.
- `max_depth=None` — không giới hạn độ sâu ở bước khởi điểm, tuning ở Tuần 4 nếu phát hiện overfit qua cross-validation.
- `class_weight='balanced'` — chuẩn bị sẵn để xử lý mất cân bằng lớp 63/37 (theo báo cáo Tuần 2), áp dụng chính thức khi huấn luyện ở Tuần 4.

SVM:

- `kernel='rbf'` làm baseline.
- `C=1.0, gamma='scale'` — mặc định scikit-learn, làm điểm khởi đầu; tuning bằng GridSearch ở Tuần 4 nếu còn thời gian.
- `class_weight='balanced'` — cùng lý do như Random Forest.

Đánh giá: cross-validation k=5 trên toàn bộ `train.csv`, không tách validation riêng — vì đây là baseline không cần theo dõi learning curve qua epoch như mô hình deep learning.

3. Chi tiết Tầng 2 — LSTM baseline

```

Input: sequence (250,) int, giá trị index theo vocab (Pipeline 2)
→ Embedding(input_dim=96, output_dim=64)
→ LSTM(units=64, return_sequences=False)
→ Dropout(rate=0.2)
→ Dense(units=1, activation='sigmoid')
Output: xác suất thuộc lớp SQLi (0-1)

```

Không dùng Dense layer trung gian giữa LSTM và Output — quyết định có chủ đích để giữ mô hình đơn giản, giảm số tham số cần học, phù hợp với bài toán phân loại nhị phân không yêu cầu nhiều tầng biến đổi ngữ nghĩa sau LSTM.

4. Chi tiết Tầng 3 — LSTM + Attention

```

Input: sequence (250,) int — giống hệt Tầng 2
→ Embedding(input_dim=96, output_dim=64)           [giống Tầng 2]
→ LSTM(units=64, return_sequences=True)             [khác Tầng 2: return_sequences=True]
→ Attention layer (additive, tự cài đặt):
    score = Dense(1, activation='tanh')(lstm_output) # (batch, 250, 1)
    weights = softmax(score, axis=1)                 # (batch, 250, 1)
    context = sum(weights * lstm_output, axis=1)      # (batch, 64)
→ Dropout(rate=0.2)                                  [giống Tầng 2]
→ Dense(units=1, activation='sigmoid')               [giống Tầng 2]
Output: xác suất (0-1) + attention_weights (250,) — lưu riêng để trực quan hóa ở Tuần 7

```

Lý do chọn additive attention tự cài đặt thay vì `tf.keras.layers.Attention` có sẵn: mục tiêu Tuần 7 là trực quan hóa trọng số attention trên từng ký tự của câu truy vấn cụ thể — additive attention cho ra trực tiếp 1 trọng số/1 vị trí thời gian (dễ vẽ heatmap), trong khi `tf.keras.layers.Attention` (dot-product, cần tự tạo query/key/value) phức tạp hơn khi diễn giải và không tự nhiên cho ra dạng trọng số đơn giản này.

Nguyên tắc bắt buộc: mọi thành phần khác ngoài Attention (Embedding, LSTM units, Dropout, Dense output, Loss, Optimizer) giữ nguyên vẹn giống Tầng 2 — đảm bảo khi so sánh kết quả ở Tuần 7, chênh lệch (nếu có) chỉ có thể quy cho việc bổ sung Attention, không lẫn với thay đổi cấu hình khác.

5. Hyperparameter huấn luyện chung (Tầng 2 và Tầng 3)

Tham số	Giá trị	Ghi chú
Batch size	64	dùng chung cho cả 2 tầng
Epoch tối đa	30	dùng chung cho cả 2 tầng
Early stopping	theo dõi <code>val_loss</code> , <code>patience=5</code>	dừng khi không cải thiện sau 5 epoch liên tiếp
Optimizer	<code>Adam(learning_rate=0.001)</code>	dùng chung, không tinh chỉnh riêng cho từng tầng ở bước này

Việc cố định các siêu tham số này trước khi code Tuần 5-6 (không phải quyết định ngẫu hứng lúc code) là điều kiện tiên quyết để kết luận so sánh ở Tuần 7 có giá trị khoa học.

6. Chỉ số đánh giá thống nhất

Áp dụng đồng nhất cho cả 3 tầng: Precision, Recall, F1-score, Confusion Matrix. Không dùng Accuracy làm chỉ số chính do lệch lớp (63/37) — Accuracy trong trường hợp này có thể gây hiểu nhầm về hiệu năng thực sự của mô hình trên lớp thiểu số (SQLi).